

GrI06

FROMENT Sacha

ABELHAJ Youssef

Projet D'IAP

Gestion d'une table de données

Année 2017 - 2018

Table des matières

Brève présentation.....page 3

Bilans de validation.....page 4 à 9

Bilan de projet.....page 10

Annexes.....page 11 à 27

Sprint 1.....page 11 à 12

Sprint 2.....page 12 à 14

Sprint 3.....page 14 à 16

Sprint 4.....page 17 à 19

Sprint 5.....page 20 à 26

Trace d'exécution.....page 27

Brève présentation

L'objet de ce projet est la programmation d'un interpréteur de commandes capable de créer et gérer des données insérées dans des tables. Les types de données pouvant être gérés par l'application sont au nombre de 4 : INT, FLOAT, TEXT et DATE. La création d'un tableau est déterminée par le nom que nous lui donnons, le nombre de champs(colonnes) que nous spécifions et le nom de ces champs accompagnés du type de donnée pour chaque champ. Il est possible de créer des lignes d'enregistrement composés d'une valeur par champ défini, le nombre maximum de champ étant de 25 et d'enregistrement de 100.

Ex : Create_table Pays 5 nom TEXT pib INT capitale TEXT latitude FLOAT longitude FLOAT

Le nombre de commandes saisissables est au nombre de 9 dont une de sortie du programme (entrées) :

Create_table	: permet de créer une table
Afficher_schema	: permet d'afficher le schéma d'une table
Insert_enregistrement	: permet l'insertion d'un enregistrement
Afficher_enregistrements	: permet l'affichage de tous les enregistrements
Delete_enregistrement	: permet la suppression d'un enregistrement
Afficher_enregistrement	: permet l'affichage d'un seul enregistrement
Select_enregistrement	: permet l'affichage d'enregistrements selon 2 critères
Delete_table	: permet la suppression d'une table
Exit	: permet de sortir du programme

En sortie de l'application, nous avons l'affichage des différents éléments demandés ou alors des messages d'information lors de la demande d'une table ou d'un enregistrement inconnu.

Lorsque l'on demande l'affichage d'un élément, celui-ci s'affichera précédé de son type :

Ex : I TEXT Turquie INT 10946 TEXT Ankara FLOAT 39.92 FLOAT 32.86

Bilans de validation

Sprint 1 :

Pour le premier sprint, l'objectif était de créer les fonctions de création d'une table et d'affichage du schéma de celle-ci. Nous saisissons le JDT suivant :

```
1. Create_table Pays 5 nom TEXT pib INT capitale TEXT latitude FLOAT longitude FLOAT
2. Create_table Pays 5 nom TEXT pib INT capitale TEXT latitude FLOAT longitude FLOAT
3. Afficher_schema Monde
4. Afficher_schema Pays
5. Exit
```

Pour obtenir comme résultat affiché :

```
1. table existante
2. table inconnue
3. Pays 5
4. nom TEXT
5. pib INT
6. capitale TEXT
7. latitude FLOAT
8. longitude FLOAT
```

On a pu déterminer d'après ces tests qu'après la création d'une table, lorsque on essayait de recréer celle-ci, on était indiqués qu'elle existait déjà.

Aussi on voit que l'affichage de « table inconnue » s'affiche bien lorsque on essaie d'afficher le schéma d'une table qui n'existe pas.

L'affichage du schéma de la table s'affiche correctement d'après les paramètres fournis lors de sa création. La fermeture du programme fonctionne également.

Sprint 2 :

Pour le second sprint, l'objectif était de créer les fonctions de création de lignes d'enregistrement et d'affichage de ces lignes. Nous saisissons le JDT suivant :

```
1. Create_table Pays 5 nom TEXT pib INT capitale TEXT latitude FLOAT longitude FLOAT
2. Afficher_schema Pays
3. Insert_enregistrement Pays Turquie 10946 Ankara 39.92 32.86
4. Insert_enregistrement Pays Chine 6807 Pekin 39.92 116.44
5. Insert_enregistrement Pays Allemagne 45085 Berlin 52.52 13.41
6. Insert_enregistrement Pays Mexique 11180 Mexico 19.42 -99.12
7. Insert_enregistrement Pays Canada 51958 Ottawa 45.41 -75.69
8. Insert_enregistrement Pays Nigeria 3006 Lagos 6.45 3.39
9. Insert_enregistrement Pays Australie 67468 Melbourne -37.81 144.96
10. Insert_enregistrement Pays Bresil 11208 Brasilia -22.45 -49.23
11. Insert_enregistrement Pays Japon 38492 Tokyo 35.70 139.73
12. Insert_enregistrement Pays Zimbabwe 906 Harare -17.82 31.03
13. Insert_enregistrement Pays Russie 14555 Moscou 55.75 37.61
14. Insert_enregistrement Pays Maroc 3083 Rabat 33.97 -6.84
15. Insert_enregistrement Pays France 41421 Paris 48.85 2.35
16. Insert_enregistrement Pays Singapour 55182 Singapour 1.35 103.81
17. Insert_enregistrement Pays Indonesie 3475 Jakarta -6.17 106.82
18. Afficher_enregistrements Pays
```

On obtenait comme résultat de sortie dans la trace d'exécution :

```

1. Pays 5
2. nom TEXT
3. pib INT
4. capitale TEXT
5. latitude FLOAT
6. longitude FLOAT
7. 1 TEXT INT 10946 TEXT Ankara FLOAT 39.92 FLOAT 32.86
8. 2 TEXT Chine INT 6807 TEXT Pekin FLOAT 39.92 FLOAT 116.44
9. 3 TEXT Allemagne INT 45085 TEXT Berlin FLOAT 52.52 FLOAT 13.41
10. 4 TEXT Mexique INT 11180 TEXT Mexico FLOAT 19.42 FLOAT -99.12
11. 5 TEXT Canada INT 51958 TEXT Ottawa FLOAT 45.41 FLOAT -75.69
12. 6 TEXT Nigeria INT 3006 TEXT Lagos FLOAT 6.45 FLOAT 3.39
13. 7 TEXT Australie INT 67468 TEXT Melbourne FLOAT -37.81 FLOAT 144.96
14. 8 TEXT Bresil INT 11208 TEXT Brasilia FLOAT -22.45 FLOAT -49.23
15. 9 TEXT Japon INT 38492 TEXT Tokyo FLOAT 35.70 FLOAT 139.73
16. 10 TEXT Zimbabwe INT 906 TEXT Harare FLOAT -17.82 FLOAT 31.03
17. 11 TEXT Russie INT 14555 TEXT Moscou FLOAT 55.75 FLOAT 37.61
18. 12 TEXT Maroc INT 3083 TEXT Rabat FLOAT 33.97 FLOAT -6.84
19. 13 TEXT France INT 41421 TEXT Paris FLOAT 48.85 FLOAT 2.35
20. 14 TEXT Singapour INT 55182 TEXT Singapour FLOAT 1.35 FLOAT 103.81
21. 15 TEXT Indonesie INT 3475 TEXT Jakarta FLOAT -6.17 FLOAT 106.82

```

On observe que l'insertion des enregistrements s'effectue correctement puisque la commande d'affichage des enregistrements a bien affiché tous ceux qui ont été déclarés.

Sprint 3 :

Pour le troisième sprint, l'objectif était de créer les fonctions d'affichage d'une ligne d'enregistrement spécifique et la suppression d'une de ces lignes. Nous saisissons le JDT suivant :

```

1. Create_table Pays 5 nom TEXT pib INT capitale TEXT latitude FLOAT longitude FLOAT
2. Afficher_schema Pays
3. Insert_enregistrement Pays Turquie 10946 Ankara 39.92 32.86
4. Insert_enregistrement Pays Chine 6807 Pekin 39.92 116.44
5. Insert_enregistrement Pays Allemagne 45085 Berlin 52.52 13.41
6. Insert_enregistrement Pays Mexique 11180 Mexico 19.42 -99.12
7. Insert_enregistrement Pays Canada 51958 Ottawa 45.41 -75.69
8. Insert_enregistrement Pays Nigeria 3006 Lagos 6.45 3.39
9. Insert_enregistrement Pays Australie 67468 Melbourne -37.81 144.96
10. Insert_enregistrement Pays Bresil 11208 Brasilia -22.45 -49.23
11. Insert_enregistrement Pays Japon 38492 Tokyo 35.70 139.73
12. Insert_enregistrement Pays Zimbabwe 906 Harare -17.82 31.03
13. Insert_enregistrement Pays Russie 14555 Moscou 55.75 37.61
14. Insert_enregistrement Pays Maroc 3083 Rabat 33.97 -6.84
15. Insert_enregistrement Pays France 41421 Paris 48.85 2.35
16. Insert_enregistrement Pays Singapour 55182 Singapour 1.35 103.81
17. Afficher_enregistrements Pays
18. Delete_enregistrement Pays 10
19. Insert_enregistrement Pays Indonesie 3475 Jakarta -6.17 106.82
20. Delete_enregistrement Pays 1
21. Afficher_enregistrement Pays 10
22. Afficher_enregistrement Pays 1
23. Exit

```

Hormis les commandes déjà vérifiées, on observe cette trace d'exécution :

```

1. Pays 5
2. nom TEXT
3. pib INT
4. capitale TEXT
5. latitude FLOAT
6. longitude FLOAT
7. 1 TEXT Turquie INT 10946 TEXT Ankara FLOAT 39.92 FLOAT 32.86
8. 2 TEXT Chine INT 6807 TEXT Pekin FLOAT 39.92 FLOAT 116.44
9. 3 TEXT Allemagne INT 45085 TEXT Berlin FLOAT 52.52 FLOAT 13.41
10. 4 TEXT Mexique INT 11180 TEXT Mexico FLOAT 19.42 FLOAT -99.12
11. 5 TEXT Canada INT 51958 TEXT Ottawa FLOAT 45.41 FLOAT -75.69
12. 6 TEXT Nigeria INT 3006 TEXT Lagos FLOAT 6.45 FLOAT 3.39
13. 7 TEXT Australie INT 67468 TEXT Melbourne FLOAT -37.81 FLOAT 144.96
14. 8 TEXT Bresil INT 11208 TEXT Brasilia FLOAT -22.45 FLOAT -49.23
15. 9 TEXT Japon INT 38492 TEXT Tokyo FLOAT 35.70 FLOAT 139.73
16. 10 TEXT Zimbabwe INT 906 TEXT Harare FLOAT -17.82 FLOAT 31.03
17. 11 TEXT Russie INT 14555 TEXT Moscou FLOAT 55.75 FLOAT 37.61
18. 12 TEXT Maroc INT 3083 TEXT Rabat FLOAT 33.97 FLOAT -6.84
19. 13 TEXT France INT 41421 TEXT Paris FLOAT 48.85 FLOAT 2.35
20. 14 TEXT Singapour INT 55182 TEXT Singapour FLOAT 1.35 FLOAT 103.81
21. 10 TEXT Maroc INT 3083 TEXT Rabat FLOAT 33.97 FLOAT -6.84
22. 1 TEXT Chine INT 6807 TEXT Pekin FLOAT 39.92 FLOAT 116.44

```

On voit bien que l'affichage de l'enregistrement I0 et I s'effectue après et séparément de l'affichage de tous les enregistrements issus de la commande « Affichage_enregistrements ». De plus on observe que l'enregistrement 2 est bien devenu le I après la suppression du premier enregistrement original. On pouvait donc valider que la fonction Afficher_enregistrement et Delete_enregistrement fonctionnaient.

Sprint 4 :

Pour le quatrième sprint l'objectif était de créer la fonction de suppression d'une table lorsqu'il y en avait déjà une de créée. Nous saisissons le JDT suivant :

```

1. Create_table Pays 5 nom TEXT pib INT capitale TEXT latitude FLOAT longitude FLOAT
2. Afficher_schema Pays
3. Insert_enregistrement Pays Turquie 10946 Ankara 39.92 32.86
4. ...
5. Insert_enregistrement Pays Singapour 55182 Singapour 1.35 103.81
6. Afficher_enregistrements Pays
7. Delete_enregistrement Pays I0
8. Insert_enregistrement Pays Indonesie 3475 Jakarta -6.17 106.82
9. Delete_enregistrement Pays 1
10. Afficher_enregistrement Pays I0
11. Delete_table Pays
12. Create_table Calendrier 3 evenement DATE description TEXT pays TEXT
13. Afficher_schema Calendrier
14. Insert_enregistrement Calendrier 14/10/1066 Bataille_Hastings Angleterre
15. Insert_enregistrement Calendrier 19/08/1274 Sacre_Edouard_I Angleterre
16. Insert_enregistrement Calendrier 07/10/1337 Guerre_Cent_Ans France
17. Insert_enregistrement Calendrier 07/06/1494 Traite_Tordesillas Espagne
18. Insert_enregistrement Calendrier 13/09/1515 Bataille_Marignan France
19. Insert_enregistrement Calendrier 10/08/1539 Ordonnance_Villers_Cotterets France

```



```
20. Afficher_enregistrement Calendrier 2
21. Insert_enregistrement Calendrier 13/04/1598 Edit_Nantes France
22. Insert_enregistrement Calendrier 02/09/1666 Incendie_Londres Angleterre
23. Insert_enregistrement Calendrier 21/01/1793 Mort_Louix_XVI France
24. Insert_enregistrement Calendrier 28/05/1936 Machine_Turing Angleterre
25. Afficher_enregistrements Calendrier
```

Hormis les commandes déjà vérifiées, on observe cette trace d'exécution :

```
1. Pays 5
2. nom TEXT
3. pib INT
4. capitale TEXT
5. latitude FLOAT
6. longitude FLOAT
7. 1 TEXT Turquie INT 10946 TEXT Ankara FLOAT 39.92 FLOAT 32.86
8. 2 TEXT Chine INT 6807 TEXT Pekin FLOAT 39.92 FLOAT 116.44
9. ...
10. 14 TEXT Singapour INT 55182 TEXT Singapour FLOAT 1.35 FLOAT 103.81
11. 10 TEXT Maroc INT 3083 TEXT Rabat FLOAT 33.97 FLOAT -6.84
12. Calendrier 3
13. evenement DATE
14. description TEXT
15. pays TEXT
16. 2 DATE 19/08/1274 TEXT Sacre_Edouard_I TEXT Angleterre
17. 1 DATE 14/10/1066 TEXT Bataille_Hastings TEXT Angleterre
18. 2 DATE 19/08/1274 TEXT Sacre_Edouard_I TEXT Angleterre
19. 3 DATE 07/10/1337 TEXT Guerre_Cent_Ans TEXT France
20. 4 DATE 07/06/1494 TEXT Traite_Tordesillas TEXT Espagne
21. 5 DATE 13/09/1515 TEXT Bataille_Marignan TEXT France
22. 6 DATE 10/08/1539 TEXT Ordonnance_Villers_Cotterets TEXT France
23. 7 DATE 13/04/1598 TEXT Edit_Nantes TEXT France
24. 8 DATE 02/09/1666 TEXT Incendie_Londres TEXT Angleterre
25. 9 DATE 21/01/1793 TEXT Mort_Louix_XVI TEXT France
26. 10 DATE 28/05/1936 TEXT Machine_Turing TEXT Angleterre
```

On remarque que la création d'une nouvelle table après la suppression de l'ancienne fonctionne puisque l'affichage du schéma de Calendrier ainsi que ses enregistrements s'effectuent correctement. La fonction peut donc être considérée comme opérationnelle.

Sprint 5 :

Pour le cinquième sprint l'objectif était de créer la fonction sélection d'enregistrements selon deux critères, pour les valeurs de type INT, des bornes numériques entières, pour les valeurs de type FLOAT, des bornes numériques décimales ou entières, pour les valeurs de type TEXT, des bornes littérales entre A et Z et pour les dates des bornes dates . Nous saisissons le JDT suivant :

```
1. Create_table Pays 5 nom TEXT pib INT capitale TEXT latitude FLOAT longitude FLOAT
2. Afficher_schema Pays
3. Insert_enregistrement Pays Turquie 10946 Ankara 39.92 32.86
4. Insert_enregistrement Pays Chine 6807 Pekin 39.92 116.44
5. Insert_enregistrement Pays Allemagne 45085 Berlin 52.52 13.41
6. Insert_enregistrement Pays Mexique 11180 Mexico 19.42 -99.12
7. Insert_enregistrement Pays Canada 51958 Ottawa 45.41 -75.69
```



```
8. Insert_enregistrement Pays Nigeria 3006 Lagos 6.45 3.39
9. Insert_enregistrement Pays Australie 67468 Melbourne -37.81 144.96
10. Insert_enregistrement Pays Bresil 11208 Brasilia -22.45 -49.23
11. Insert_enregistrement Pays Japon 38492 Tokyo 35.70 139.73
12. Insert_enregistrement Pays Zimbabwe 906 Harare -17.82 31.03
13. Insert_enregistrement Pays Russie 14555 Moscou 55.75 37.61
14. Insert_enregistrement Pays Maroc 3083 Rabat 33.97 -6.84
15. Insert_enregistrement Pays France 41421 Paris 48.85 2.35
16. Insert_enregistrement Pays Singapour 55182 Singapour 1.35 103.81
17. Afficher_enregistrements Pays
18. Delete_enregistrement Pays 10
19. Insert_enregistrement Pays Indonesie 3475 Jakarta -6.17 106.82
20. Delete_enregistrement Pays 1
21. Afficher_enregistrement Pays 10
22. Select_enregistrement Pays longitude -45 45
23. Delete_table Pays
24. Create_table Calendrier 3 evenement DATE description TEXT pays TEXT
25. Afficher_schema Calendrier
26. Insert_enregistrement Calendrier 14/10/1066 Bataille_Hastings Angleterre
27. Insert_enregistrement Calendrier 19/08/1274 Sacre_Edouard_I Angleterre
28. Insert_enregistrement Calendrier 07/10/1337 Guerre_Cent_Ans France
29. Insert_enregistrement Calendrier 07/06/1494 Traite_Tordesillas Espagne
30. Insert_enregistrement Calendrier 13/09/1515 Bataille_Marignan France
31. Insert_enregistrement Calendrier 10/08/1539 Ordonnance_Villers_Cotterets France
32. Insert_enregistrement Calendrier 13/04/1598 Edit_Nantes France
33. Insert_enregistrement Calendrier 02/09/1666 Incendie_Londres Angleterre
34. Insert_enregistrement Calendrier 21/01/1793 Mort_Louix_XVI France
35. Insert_enregistrement Calendrier 28/05/1936 Machine_Turing Angleterre
36. Select_enregistrement Calendrier pays D G
37. Select_enregistrement Calendrier evenement 07/10/1337 10/08/1539
38. Exit
```

on observe cette sortie :

```
1. Pays 5
2. nom TEXT
3. pib INT
4. capitale TEXT
5. latitude FLOAT
6. longitude FLOAT
7. 1 TEXT Turquie INT 10946 TEXT Ankara FLOAT 39.92 FLOAT 32.86
8. 2 TEXT Chine INT 6807 TEXT Pekin FLOAT 39.92 FLOAT 116.44
9. 3 TEXT Allemagne INT 45085 TEXT Berlin FLOAT 52.52 FLOAT 13.41
10. 4 TEXT Mexique INT 11180 TEXT Mexico FLOAT 19.42 FLOAT -99.12
11. 5 TEXT Canada INT 51958 TEXT Ottawa FLOAT 45.41 FLOAT -75.69
12. 6 TEXT Nigeria INT 3006 TEXT Lagos FLOAT 6.45 FLOAT 3.39
13. 7 TEXT Australie INT 67468 TEXT Melbourne FLOAT -37.81 FLOAT 144.96
14. 8 TEXT Bresil INT 11208 TEXT Brasilia FLOAT -22.45 FLOAT -49.23
15. 9 TEXT Japon INT 38492 TEXT Tokyo FLOAT 35.70 FLOAT 139.73
16. 10 TEXT Zimbabwe INT 906 TEXT Harare FLOAT -17.82 FLOAT 31.03
17. 11 TEXT Russie INT 14555 TEXT Moscou FLOAT 55.75 FLOAT 37.61
18. 12 TEXT Maroc INT 3083 TEXT Rabat FLOAT 33.97 FLOAT -6.84
19. 13 TEXT France INT 41421 TEXT Paris FLOAT 48.85 FLOAT 2.35
20. 14 TEXT Singapour INT 55182 TEXT Singapour FLOAT 1.35 FLOAT 103.81
21. 10 TEXT Maroc INT 3083 TEXT Rabat FLOAT 33.97 FLOAT -6.84
22. 2 TEXT Allemagne INT 45085 TEXT Berlin FLOAT 52.52 FLOAT 13.41
23. 5 TEXT Nigeria INT 3006 TEXT Lagos FLOAT 6.45 FLOAT 3.39
24. 9 TEXT Russie INT 14555 TEXT Moscou FLOAT 55.75 FLOAT 37.61
25. 10 TEXT Maroc INT 3083 TEXT Rabat FLOAT 33.97 FLOAT -6.84
26. 11 TEXT France INT 41421 TEXT Paris FLOAT 48.85 FLOAT 2.35
27. Calendrier 3
28. evenement DATE
29. description TEXT
30. pays TEXT
```




31.	3	DATE	07/10/1337	TEXT	Guerre_Cent_Ans	TEXT	France
32.	4	DATE	07/06/1494	TEXT	Traite_Tordesillas	TEXT	Espagne
33.	5	DATE	13/09/1515	TEXT	Bataille_Marignan	TEXT	France
34.	6	DATE	10/08/1539	TEXT	Ordonnance_Villers_Cotterets	TEXT	France
35.	7	DATE	13/04/1598	TEXT	Edit_Nantes	TEXT	France
36.	9	DATE	21/01/1793	TEXT	Mort_Louix_XVI	TEXT	France
37.	3	DATE	07/10/1337	TEXT	Guerre_Cent_Ans	TEXT	France
38.	4	DATE	07/06/1494	TEXT	Traite_Tordesillas	TEXT	Espagne
39.	5	DATE	13/09/1515	TEXT	Bataille_Marignan	TEXT	France
40.	6	DATE	10/08/1539	TEXT	Ordonnance_Villers_Cotterets	TEXT	France

Les enregistrements affichés à la suite de la commande respectent bien les critères donnés, aussi bien pour ceux fournis de base que ceux que nous avons testé de notre côté. Nous avons donc conclu que la fonction était correctement implémentée.

.....

Bilan de projet

Lorsque nous avons démarré le projet, c'est-à-dire le Sprint 1, nous sommes partis de la supposition que le programme devait autoriser la création de multiples tables simultanément. Nous avons donc commencé avec une difficulté certaine notre projet à cause de cette erreur. Cependant, les premiers avancements que nous avons fait lors de la programmation de ce premier sprint erroné nous a grandement aidé lorsque nous avons dû poursuivre la programmation correcte du sprint 1. Le sprint 1 a donc pu être complété sans grande difficulté. J'ai procédé à de significantes améliorations au fil du projet ce qui m'a permis d'améliorer sa simplicité et son efficacité. Nous pouvons conclure que ce sprint (voir annexe) est réussi et que nous avons déjà bien amélioré son fonctionnement comparé à notre première réalisation.

Le Sprint 2 n'a posé aucune difficulté puisque l'implémentation non demandée que nous avons réalisé dans le sprint 1 nous a permis de comprendre rapidement la procédure à suivre pour stocker plusieurs enregistrements ainsi que de pouvoir les distinguer et les reconnaître lors de leurs appels. De plus nous avons remédié à sa concision d'avance et réduit sa taille assez rapidement dans son développement. Nous avons conclu que ce sprint était amplement réussi.

Pour le Sprint 3, la plus grande difficulté était de trouver comment supprimer un enregistrement sans avoir d'erreur d'affichage ou d'accès mémoire par la suite. Nous n'avons tout de même eu que peu de soucis pour venir à bout de cette étape. Nous avons procédé à de très nombreux test sur cette fonction pour être certains que, peu importe l'ordre de saisie des commandes, cette fonction ne pouvait pas accéder à de valeurs non voulues.

Le Sprint 4 a été le plus simple à terminer puisqu'il a suffi de 10mns pour qu'il soit complété et vérifié du fait de la simplicité de l'implémentation de la fonction `delete_table`. En effet, il était suffisant de remettre certains compteurs à 0 et d'effacer le nom de la table existante.

Le Sprint final 5 n'a pas non plus de problème. Nous avons pris une certaine habitude de séparer les différentes tâches qui devaient être effectués ce qui permettait de bien visualiser quels étaient les étapes dans la rédaction de l'algorithme. Le découpage de la fonction `compare_enregistrement` selon les types nous est immédiatement venu à l'esprit ce qui nous a permis de passer très vite à la transformation des formats des dates, la partie nous ayant le plus pris de temps pour ce Sprint. Nous avons donc bien amélioré notre capacité d'analyse et de passage d'un algorithme en langage naturel en code.

Annexes

Le Sprint 5 contient tous les commentaires. Pour un souci de clarté, les sprints 1 à 4 ne contiendront pas de commentaires.

Sprint 1 (sans commentaires) :

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #pragma warning(disable:4996)
5.
6. #define max_champs 25
7. #define lgMot 30
8. #define lgMax 80
9.
10. char mot[lgMot + 1];
11. unsigned int cmp_table = 0;
12.
13. typedef struct {
14.     char nom[lgMot + 1];
15.     char type[lgMot + 1];
16. } Champ;
17. typedef struct {
18.     char nom[lgMot + 1];
19.     Champ schema[max_champs];
20.     unsigned int nbChamps;
21. }Table;
22.
23. void create_table(Table *t);
24.
25. void afficher_schema(const Table *t);
26.
27. int main() {
28.     Table table = { NULL };
29.     while (1) {
30.
31.         scanf("%s", &mot);
32.
33.         if (strcmp(mot, "Create_table") == 0) {
34.             create_table(&table);
35.         }
36.         else if (strcmp(mot, "Afficher_schema") == 0) {
37.             afficher_schema(&table);
38.         }
39.         else if (strcmp(mot, "Exit") == 0) {
40.             exit(0);
41.         }
42.     }
43.     system("pause");
44.     return 0;
45. }
46. void create_table(Table *t) {
47.     unsigned char temp[lgMot + 1];
48.     scanf("%s", &temp);
49.     if (strcmp(temp, t->nom) != 0 && cmp_table != 0) {
50.         printf("table inconnue\n");
51.     }
52.     else if (cmp_table != 0)
53.         printf("table existante\n");
54.     else {
55.         scanf("%u", &(t->nbChamps));
56.         strcpy(t->nom, temp);
```



```
57.     for (unsigned int i = 0; i < t->nbChamps; ++i) {
58.         scanf("%s %s", t->schema[i].nom, t->schema[i].type);
59.     }
60.     ++cmp_table;
61. }
62. }
63. void afficher_schema(const Table *t) {
64.     unsigned char table_nom[lgMot + 1];
65.     scanf("%s", &table_nom);
66.     if (strcmp(table_nom, t->nom) == 0) {
67.         printf("%s %u\n", &t->nom, t->nbChamps);
68.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
69.             printf("%s %s\n", t->schema[i].nom, t->schema[i].type);
70.         }
71.     }
72.     else {
73.         printf("table inconnue\n");
74.     }
75. }
```

Sprint 2 (sans commentaires) :

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #pragma warning(disable:4996)
5.
6. #define max_champs 25
7. #define max_enregistrement 100
8. #define lgMot 30
9. #define lgMax 80
10.
11. char mot[lgMot + 1];
12. unsigned int cmp_table = 0;
13.
14. typedef struct {
15.     unsigned char data[lgMot + 1];
16. }Data;
17. typedef struct {
18.     Data colonne[max_champs];
19. }Enregistrement;
20. typedef struct {
21.     char nom[lgMot + 1];
22.     char type[lgMot + 1];
23. } Champ;
24. typedef struct {
25.     char nom[lgMot + 1];
26.     Champ schema[max_champs];
27.     Enregistrement enrg[max_enregistrement];
28.     unsigned int nbChamps;
29.     unsigned int nbEnrg;
30. }Table;
31.
32. void create_table(Table *t);
33. void afficher_schema(const Table *t);
34. void inserer_enregistrement(Table* t);
35. void afficher_enregistrements(const Table* t);
36.
37. int main() {
38.     Table table = { NULL };
39.     table.nbEnrg = 0;
40.     while (1) {
41.
42.         scanf("%s", &mot);
43.     }
```



```
44.     if (strcmp(mot, "Create_table") == 0) {
45.         create_table(&table);
46.     }
47.     else if (strcmp(mot, "Afficher_schema") == 0) {
48.         afficher_schema(&table);
49.     }
50.     else if (strcmp(mot, "Insert_enregistrement") == 0) {
51.         inserer_enregistrement(&table);
52.     }
53.     else if (strcmp(mot, "Afficher_enregistrements") == 0) {
54.         afficher_enregistrements(&table);
55.     }
56.     else if (strcmp(mot, "Exit") == 0) {
57.         exit(0);
58.     }
59. }
60. system("pause");
61. return 0;
62. }
63. void create_table(Table *t) {
64.     unsigned char temp[lgMot + 1];
65.     scanf("%s", &temp);
66.     if (strcmp(temp, t->nom) != 0 && cmp_table != 0) {
67.         printf("table inconnue\n");
68.     }
69.     else if (cmp_table != 0)
70.         printf("table existante\n");
71.     else {
72.         scanf("%u", &(t->nbChamps));
73.         strcpy(t->nom, temp);
74.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
75.             scanf("%s %s", t->schema[i].nom, t->schema[i].type);
76.         }
77.         ++cmp_table;
78.     }
79. }
80. void afficher_schema(const Table *t) {
81.     unsigned char table_nom[lgMot + 1];
82.     scanf("%s", &table_nom);
83.     if (strcmp(table_nom, t->nom) == 0) {
84.         printf("%s %u\n", &t->nom, t->nbChamps);
85.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
86.             printf("%s %s\n", t->schema[i].nom, t->schema[i].type);
87.         }
88.     }
89.     else {
90.         printf("table inconnue\n");
91.     }
92. }
93. void inserer_enregistrement(Table* t) {
94.     unsigned char table_nom[lgMot + 1];
95.     scanf("%s", &table_nom);
96.     if (strcmp(table_nom, t->nom) == 0) {
97.         ++t->nbEnrg;
98.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
99.             scanf("%s", t->enrg[t->nbEnrg].colonne[i].data);
100.        }
101.    }
102.    else {
103.        printf("table inconnue\n");
104.    }
105. }
106. void afficher_enregistrements(const Table* t) {
107.     unsigned char table_nom[lgMot + 1];
108.     scanf("%s", &table_nom);
```



```
109.     if (strcmp(table_nom, t->nom) == 0) {
110.         for (unsigned int i = 1; i <= t->nbEnrg; ++i) {
111.             printf("%u", i);
112.             for (unsigned int n = 0; n < t->nbChamps; ++n) {
113.                 printf(" %s %s", t->schema[n].type, t->enrg[i].colonne[n].data);
114.             }
115.             printf("\n");
116.         }
117.     }
118.     else {
119.         printf("table inconnue\n");
120.     }
121. }
```

Sprint 3 (sans commentaires) :

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #pragma warning(disable:4996)
5.
6. #define max_champs 25
7. #define max_enregistrement 100
8. #define lgMot 30
9. #define lgMax 80
10.
11. char mot[lgMot + 1];
12. unsigned int cmp_table = 0;
13.
14. typedef struct {
15.     unsigned char data[lgMot + 1];
16. }Data;
17. typedef struct {
18.     Data colonne[max_champs];
19. }Enregistrement;
20. typedef struct {
21.     char nom[lgMot + 1];
22.     char type[lgMot + 1];
23. } Champ;
24. typedef struct {
25.     char nom[lgMot + 1];
26.     Champ schema[max_champs];
27.     Enregistrement enrg[max_enregistrement];
28.     unsigned int nbChamps;
29.     unsigned int nbEnrg;
30. }Table;
31.
32. void create_table(Table *t);
33. void afficher_schema(const Table *t);
34. void inserer_enregistrement(Table* t);
35. void afficher_enregistrements(const Table* t);
36. void delete_enregistrement(Table* t);
37. void afficher_enregistrement(const Table* t);
38.
39. int main() {
40.     Table table = { NULL };
41.     table.nbEnrg = 0;
42.     while (1) {
43.
44.         scanf("%s", &mot);
45.
46.         if (strcmp(mot, "Create_table") == 0) {
47.             create_table(&table);
48.         }
49.         else if (strcmp(mot, "Afficher_schema") == 0) {
```



```
50.     afficher_schema(&table);
51. }
52. else if (strcmp(mot, "Insert_enregistrement") == 0) {
53.     inserer_enregistrement(&table);
54. }
55. else if (strcmp(mot, "Afficher_enregistrements") == 0) {
56.     afficher_enregistrements(&table);
57. }
58. else if (strcmp(mot, "Delete_enregistrement") == 0) {
59.     delete_enregistrement(&table);
60. }
61. else if (strcmp(mot, "Afficher_enregistrement") == 0) {
62.     afficher_enregistrement(&table);
63. }
64. else if (strcmp(mot, "Exit") == 0) {
65.     exit(0);
66. }
67. }
68. system("pause");
69. return 0;
70. }
71. void create_table(Table *t) {
72.     unsigned char temp[lgMot + 1];
73.     scanf("%s", &temp);
74.     if (strcmp(temp, t->nom) != 0 && cmp_table != 0) {
75.         printf("table inconnue\n");
76.     }
77.     else if (cmp_table != 0)
78.         printf("table existante\n");
79.     else {
80.         scanf("%u", &(t->nbChamps));
81.         strcpy(t->nom, temp);
82.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
83.             scanf("%s %s", t->schema[i].nom, t->schema[i].type);
84.         }
85.         ++cmp_table;
86.     }
87. }
88. void afficher_schema(const Table *t) {
89.     unsigned char table_nom[lgMot + 1];
90.     scanf("%s", &table_nom);
91.     if (strcmp(table_nom, t->nom) == 0) {
92.         printf("%s %u\n", &t->nom, t->nbChamps);
93.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
94.             printf("%s %s\n", t->schema[i].nom, t->schema[i].type);
95.         }
96.     }
97.     else {
98.         printf("table inconnue\n");
99.     }
100. }
101. void inserer_enregistrement(Table* t) {
102.     unsigned char table_nom[lgMot + 1];
103.     scanf("%s", &table_nom);
104.     if (strcmp(table_nom, t->nom) == 0) {
105.         ++t->nbEnrg;
106.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
107.             scanf("%s", t->enrg[t->nbEnrg].colonne[i].data);
108.         }
109.     }
110.     else {
111.         printf("table inconnue\n");
112.     }
113. }
114. void afficher_enregistrements(const Table* t) {
```



```
115.     unsigned char table_nom[lgMot + 1];
116.     scanf("%s", &table_nom);
117.     if (strcmp(table_nom, t->nom) == 0) {
118.         for (unsigned int i = 1; i <= t->nbEnrg; ++i) {
119.             printf("%u", i);
120.             for (unsigned int n = 0; n < t->nbChamps; ++n) {
121.                 printf(" %s %s", t->schema[n].type, t->enrg[i].colonne[n].data);
122.             }
123.             printf("\n");
124.         }
125.     }
126.     else {
127.         printf("table inconnue\n");
128.     }
129. }
130. void delete_enregistrement(Table* t) {
131.     unsigned char table_nom[lgMot + 1];
132.     unsigned int rang_suppr;
133.     scanf("%s", &table_nom);
134.     if (strcmp(table_nom, t->nom) == 0) {
135.         scanf("%u", &rang_suppr);
136.         for (; rang_suppr <= t->nbEnrg; ++rang_suppr) {
137.             for (unsigned int i = 0; i < t->nbChamps; ++i) {
138.                 if (rang_suppr == t->nbEnrg) {
139.                     --t->nbEnrg;
140.                     return;
141.                 }
142.                 else {
143.                     strcpy(t->enrg[rang_suppr].colonne[i].data, t-
>enrg[rang_suppr + 1].colonne[i].data);
144.                 }
145.             }
146.         }
147.         printf("enregistrement inconnu\n");
148.         return;
149.     }
150.     else
151.         printf("table inconnue\n");
152. }
153. void afficher_enregistrement(const Table* t) {
154.     unsigned char table_nom[lgMot + 1];
155.     unsigned int rang_enrg;
156.     scanf("%s", &table_nom);
157.     if (strcmp(table_nom, t->nom) == 0) {
158.         scanf("%u", &rang_enrg);
159.         if (rang_enrg <= t->nbEnrg) {
160.             printf("%u", rang_enrg);
161.             for (unsigned int i = 0; i < t->nbChamps; ++i) {
162.                 printf(" %s %s", t->schema[i].type, t-
>enrg[rang_enrg].colonne[i].data);
163.             }
164.             printf("\n"); return;
165.         }
166.         printf("enregistrement inconnu\n");
167.         return;
168.     }
169.     else
170.         printf("table inconnue\n");
171. }
```




Sprint 4 (sans commentaires) :

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #pragma warning(disable:4996)
5.
6. #define max_champs 25
7. #define max_enregistrement 100
8. #define lgMot 30
9. #define lgMax 80
10.
11. char mot[lgMot + 1];
12. unsigned int cmp_table = 0;
13.
14. typedef struct {
15.     unsigned char data[lgMot + 1];
16. }Data;
17. typedef struct {
18.     Data colonne[max_champs];
19. }Enregistrement;
20. typedef struct {
21.     char nom[lgMot + 1];
22.     char type[lgMot + 1];
23. } Champ;
24. typedef struct {
25.     char nom[lgMot + 1];
26.     Champ schema[max_champs];
27.     Enregistrement enrg[max_enregistrement];
28.     unsigned int nbChamps;
29.     unsigned int nbEnrg;
30. }Table;
31.
32. void create_table(Table *t);
33. void afficher_schema(const Table *t);
34. void inserer_enregistrement(Table* t);
35. void afficher_enregistrements(const Table* t);
36. void delete_enregistrement(Table* t);
37. void afficher_enregistrement(const Table* t);
38. void delete_table(Table* t);
39.
40. int main() {
41.     Table table = { NULL };
42.     table.nbEnrg = 0;
43.     while (1) {
44.
45.         scanf("%s", &mot);
46.
47.         if (strcmp(mot, "Create_table") == 0) {
48.             create_table(&table);
49.         }
50.         else if (strcmp(mot, "Afficher_schema") == 0) {
51.             afficher_schema(&table);
52.         }
53.         else if (strcmp(mot, "Insert_enregistrement") == 0) {
54.             inserer_enregistrement(&table);
55.         }
56.         else if (strcmp(mot, "Afficher_enregistrements") == 0) {
57.             afficher_enregistrements(&table);
58.         }
59.         else if (strcmp(mot, "Delete_enregistrement") == 0) {
60.             delete_enregistrement(&table);
61.         }
62.         else if (strcmp(mot, "Afficher_enregistrement") == 0) {
63.             afficher_enregistrement(&table);
```



```
64.     }
65.     else if (strcmp(mot, "Delete_table") == 0) {
66.         delete_table(&table);
67.     }
68.     else if (strcmp(mot, "Exit") == 0) {
69.         exit(0);
70.     }
71. }
72. system("pause");
73. return 0;
74. }
75. void create_table(Table *t) {
76.     unsigned char temp[lgMot + 1];
77.     scanf("%s", &temp);
78.     if (strcmp(temp, t->nom) != 0 && cmp_table != 0) {
79.         printf("table inconnue\n");
80.     }
81.     else if (cmp_table != 0)
82.         printf("table existante\n");
83.     else {
84.         scanf("%u", &(t->nbChamps));
85.         strcpy(t->nom, temp);
86.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
87.             scanf("%s %s", t->schema[i].nom, t->schema[i].type);
88.         }
89.         ++cmp_table;
90.     }
91. }
92. void afficher_schema(const Table *t) {
93.     unsigned char table_nom[lgMot + 1];
94.     scanf("%s", &table_nom);
95.     if (strcmp(table_nom, t->nom) == 0) {
96.         printf("%s %u\n", &t->nom, t->nbChamps);
97.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
98.             printf("%s %s\n", t->schema[i].nom, t->schema[i].type);
99.         }
100.    }
101.    else {
102.        printf("table inconnue\n");
103.    }
104. }
105. void inserer_enregistrement(Table* t) {
106.     unsigned char table_nom[lgMot + 1];
107.     scanf("%s", &table_nom);
108.     if (strcmp(table_nom, t->nom) == 0) {
109.         ++t->nbEnrg;
110.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
111.             scanf("%s", t->enrg[t->nbEnrg].colonne[i].data);
112.         }
113.     }
114.     else {
115.         printf("table inconnue\n");
116.     }
117. }
118. void afficher_enregistrements(const Table* t) {
119.     unsigned char table_nom[lgMot + 1];
120.     scanf("%s", &table_nom);
121.     if (strcmp(table_nom, t->nom) == 0) {
122.         for (unsigned int i = 1; i <= t->nbEnrg; ++i) {
123.             printf("%u", i);
124.             for (unsigned int n = 0; n < t->nbChamps; ++n) {
125.                 printf(" %s %s", t->schema[n].type, t->enrg[i].colonne[n].data);
126.             }
127.             printf("\n");
128.         }
129.     }
```



```
129.     }
130.     else {
131.         printf("table inconnue\n");
132.     }
133. }
134. void delete_enregistrement(Table* t) {
135.     unsigned char table_nom[lgMot + 1];
136.     unsigned int rang_suppr;
137.     scanf("%s", &table_nom);
138.     if (strcmp(table_nom, t->nom) == 0) {
139.         scanf("%u", &rang_suppr);
140.         for (; rang_suppr <= t->nbEnrg; ++rang_suppr) {
141.             for (unsigned int i = 0; i < t->nbChamps; ++i) {
142.                 if (rang_suppr == t->nbEnrg) {
143.                     --t->nbEnrg;
144.                     return;
145.                 }
146.                 else {
147.                     strcpy(t->enrg[rang_suppr].colonne[i].data, t-
>enrg[rang_suppr + 1].colonne[i].data);
148.                 }
149.             }
150.         }
151.         printf("enregistrement inconnu\n");
152.         return;
153.     }
154.     else
155.         printf("table inconnue\n");
156. }
157. void afficher_enregistrement(const Table* t) {
158.     unsigned char table_nom[lgMot + 1];
159.     unsigned int rang_enrg;
160.     scanf("%s", &table_nom);
161.     if (strcmp(table_nom, t->nom) == 0) {
162.         scanf("%u", &rang_enrg);
163.         if (rang_enrg <= t->nbEnrg) {
164.             printf("%u", rang_enrg);
165.             for (unsigned int i = 0; i < t->nbChamps; ++i) {
166.                 printf(" %s %s", t->schema[i].type, t-
>enrg[rang_enrg].colonne[i].data);
167.             }
168.             printf("\n"); return;
169.         }
170.         printf("enregistrement inconnu\n");
171.         return;
172.     }
173.     else
174.         printf("table inconnue\n");
175. }
176. void delete_table(Table* t) {
177.     unsigned char table_nom[lgMot + 1];
178.     scanf("%s", &table_nom);
179.     if (strcmp(table_nom, t->nom) == 0) {
180.         cmp_table = t->nbEnrg = t->nbChamps = 0;
181.         strcpy(t->nom, "\0");
182.     }
183.     else {
184.         printf("table inconnue\n");
185.     }
186. }
```

Sprint 5 (avec commentaires) :

```

1.  /*Sprint_5.c
2.  *FROMENT et ABELHAJ
3.  *Groupe 106
4.  *Créé le 13 octobre 2017
5.  */
6.  #include <stdio.h>
7.  #include <stdlib.h>
8.  #include <string.h>
9.  #pragma warning(disable:4996)
10.
11. #define max_champs 25
12. // Nombre maximum de champs d'une table
13. #define max_enregistrement 100
14. //Nombre maximal d'enregistrements
15. #define lgMot 30
16. // Longueur max d'une chaîne de caracteres
17. #define lgMax 80
18. // Longueur maximale d'une ligne de commande
19.
20. typedef enum { FALSE, TRUE }bool;
21. char mot[lgMot + 1];
22. //une chaîne de caractères dans laquelle on stocke la commande saisie
23. unsigned char crit1[lgMot + 1], crit2[lgMot + 1];
24. // les critères saisies pour select_enregistrement()
25. unsigned int cmp_table = 0;
26. //un compteur qui passe à 1 lorsque une table est créée et à 0 quand il n'y a soit pas d
   e table soit qu'elle est supprimée
27.
28. typedef struct {
29.     unsigned char data[lgMot + 1];
30.     //variable nom de type char et de longueur 30 + 1
31. }Data;
32. typedef struct {
33.     Data colonne[max_champs];
34.     //struct dans lequel on stocke les valeurs de chaque champ dans une ligne d'enregist
   rement
35. }Enregistrement;
36. typedef struct {
37.     char nom[lgMot + 1];
38.     // variable nom de type char et de longueur 30 + 1
39.     char type[lgMot + 1];
40.     // variable type de type char et de longueur 30 + 1
41. } Champ;
42. typedef struct {
43.     char nom[lgMot + 1];
44.     //chaîne de caractères dans laquelle on stocke le nom de la table
45.     Champ schema[max_champs];
46.     //struct qui contient le nom et le type de chaque champs
47.     Enregistrement enrg[max_enregistrement];
48.     //struct dans lequel on stocke les enregistrements
49.     unsigned int nbChamps;
50.     //variable indiquant le nombre de champs d'une table
51.     unsigned int nbEnrg;
52.     //variable indiquant le nombre d'enregistrement d'une table
53. }Table;
54.
55.
56. //Prototypes des fonctions utilisées
57. /*Création d'une table
58. *[in out] t la table créée
59. */
60. void create_table(Table *t);
61. /*Affichage d'une table

```



```
62. *[in] t la table à afficher
63. */
64. void afficher_schema(const Table* t);
65. /*Insertion d'une ligne d'enregistrement
66. *[in out] t la table dans laquelle on insère l'enregistrement
67. */
68. void inserer_enregistrement(Table* t);
69. /*Affichage de tout les enregistrements
70. *[in] t la table qui contient les enregistrements à afficher
71. */
72. void afficher_enregistrements(const Table* t);
73. /*Suppression d'un enregistrement
74. *[in out] t la table dans laquelle on veut supprimer un enregistrement
75. */
76. void delete_enregistrement(Table* t);
77. /*Affichage d'un enregistrement spécifique à une ligne n <= nbEnrg
78. *[in] t la table qui contient l'enregistrement à afficher
79. */
80. void afficher_enregistrement(const Table* t);
81. /*Suppression d'une table
82. *[in out] t la table à supprimer
83. */
84. void delete_table(Table* t);
85. /*Affichage de valeurs d'un champ de chaque enregistrement selon deux critères
86. *[in out] t la table contenant les enregistrements
87. */
88. void select_enregistrement(const Table* t);
89. /*Comparaison des valeurs d'enregistrements aux critères
90. *[in] type, le type de valeurs du champ choisi
91. *[in] data, la valeur à comparer aux critères
92. *[retour] la valeur FALSE(0) ou TRUE(1), déterminant la validité ou non d'un critère
93. */
94. bool compare_enregistrement(const char type[lgMot + 1], const char data[lgMot + 1]); //fonction de vérification de validité des enregistrements par rapport aux critères
95.
96.
97. int main() {
98.     Table table = { NULL };
99.     //mise à vide de la table
100.    table.nbEnrg = 0;
101.    // mise à zéro du nombre d'enregistrement
102.    while (1) {
103.        // boucle infinie sur les 9 commandes
104.        scanf("%s", &mot);
105.        // le mot lu est une chaîne de 30 caract.
106.        //on vérifie quelle est la commande saisie
107.        if (strcmp(mot, "Create_table") == 0) {
108.            create_table(&table);
109.            //créer une table
110.        }
111.        else if (strcmp(mot, "Afficher_schema") == 0) {
112.            afficher_schema(&table);
113.            //affiche le schema d'une table
114.        }
115.        else if (strcmp(mot, "Insert_enregistrement") == 0) {
116.            inserer_enregistrement(&table);
117.            //insère un enregistrement
118.        }
119.        else if (strcmp(mot, "Afficher_enregistrements") == 0) {
120.            afficher_enregistrements(&table);
121.            //affiche tous les enregistrements d'une table donnée
122.        }
123.        else if (strcmp(mot, "Delete_enregistrement") == 0) {
124.            delete_enregistrement(&table);
125.            //supprime un enregistrement
```



```
126.     }
127.     else if (strcmp(mot, "Afficher_enregistrement") == 0) {
128.         afficher_enregistrement(&table);
129.         //affiche d'un enregistrement donné
130.     }
131.     else if (strcmp(mot, "Delete_table") == 0) {
132.         delete_table(&table);
133.         //supprime tous les elements d'une table
134.     }
135.     else if (strcmp(mot, "Select_enregistrement") == 0) {
136.         select_enregistrement(&table);
137.         //selectionne des valeurs d'enregistrement selon deux critères
138.     }
139.     else if (strcmp(mot, "Exit") == 0) {
140.         exit(0);
141.         // sortie du programme principal
142.     }
143. }
144. system("pause");
145. return 0;
146. }
147. void create_table(Table *t) {
148.     unsigned char temp[lgMot + 1];
149.     //variable de stockage temporaire pour le nom de la table
150.     scanf("%s", &temp);
151.     //on demande la saisie du nom de la table
152.     if (strcmp(temp, t->nom) != 0 && cmp_table != 0) {
153.         //si le nom de la table n'existe pas et qu'on a déjà créé une table
154.         printf("table inconnue\n");
155.         //on affiche que la table demandée n'existe pas
156.     }
157.     else if (cmp_table != 0)
158.         //si la table existe déjà
159.         printf("table existante\n");
160.     else {
161.         //la première fois qu'on entre dans la fonction create_table
162.         scanf("%u", &(t->nbChamps));
163.         //on saisit le nombre de champs
164.         strcpy(t->nom, temp);
165.         //on affecte la valeur de temp dans table.nom
166.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
167.             // pour i allant de 0 à nombre de champs-1
168.             scanf("%s %s", t->schema[i].nom, t->schema[i].type);
169.             //on demande la saisie du nom et du type de chaque champs
170.         }
171.         ++cmp_table;
172.         // on incrémente le compteur de table pour indiquer son existence
173.     }
174. }
175. void afficher_schema(const Table *t) {
176.     unsigned char table_nom[lgMot + 1];
177.     //variable locale de stockage du nom de la table dont on veut afficher le schema
178.     scanf("%s", &table_nom);
179.     //on demande la saisie du nom de la table
180.     if (strcmp(table_nom, t->nom) == 0) {
181.         //on compare le nom de la table à celui saisi pour vérifier qu'elle existe
182.         printf("%s %u\n", &t->nom, t->nbChamps);
183.         //on affiche son nom et son nombre de champs
184.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
185.             // pour i allant de 0 à nombre de champs-1
186.             printf("%s %s\n", t->schema[i].nom, t->schema[i].type);
187.             //on affiche le nom et le type de chaque champs
188.         }
189.     }
```



```
189.     }
190.     else {
191.         printf("table inconnue\n");
192.         //sinon on affiche "table inconnue"
193.     }
194. }
195. void inserer_enregistrement(Table* t) {
196.     unsigned char table_nom[lgMot + 1];
197.     //variable locale de stockage du nom de la table dans laquelle on veut insérer u
n enregistrement
198.     scanf("%s", &table_nom);
199.     //on demande la saisie du nom de la table
200.     if (strcmp(table_nom, t->nom) == 0) {
201.         //on compare le nom de la table à celui saisi pour vérifier qu'elle existe
202.         ++t->nbEnrg;
203.         //on incrémente de 1 le nombre d'enregistrement
204.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
205.             // pour i allant de 0 à nombre de champs-1
206.             scanf("%s", t->enrg[t->nbEnrg].colonne[i].data);
207.             //on demande la saisie de chaque champ pour cet ligne d'enregistrement
208.         }
209.     }
210.     else {
211.         printf("table inconnue\n");
212.         //sinon on affiche "table inconnue"
213.     }
214. }
215. void afficher_enregistrements(const Table* t) {
216.     unsigned char table_nom[lgMot + 1];
217.     //variable locale de stockage du nom de la table dont on veut afficher les enreg
istrement
218.     scanf("%s", &table_nom);
219.     //on demande la saisie du nom de la table
220.     if (strcmp(table_nom, t->nom) == 0) {
221.         //on compare le nom de la table à celui saisi pour vérifier qu'elle existe
222.         for (unsigned int i = 1; i <= t->nbEnrg; ++i) {
223.             // pour i allant de 1 au nombre d'enregistrement saisi
224.             printf("%u", i);
225.             // on affiche le rang de chaque enregistrement
226.             for (unsigned int n = 0; n < t->nbChamps; ++n) {
227.                 // pour n allant de 0 à nombre de champs-1
228.                 printf(" %s %s", t->schema[n].type, t->enrg[i].colonne[n].data);
229.                 // on affiche chaque enregistrements précédés de leur type
230.             }
231.             printf("\n");
232.             //puis on revient à la ligne après l'affichage d'une ligne entière
233.         }
234.     }
235.     else {
236.         printf("table inconnue\n");
237.         //sinon on affiche "table inconnue"
238.     }
239. }
240. void delete_enregistrement(Table* t) {
241.     unsigned char table_nom[lgMot + 1];
242.     //variable locale de stockage du nom de la table dont on veut supprimer un enreg
istrement
243.     unsigned int rang_suppr;
244.     //variable locale qui indique le rang de l'enregistrement à supprimer
245.     scanf("%s", &table_nom);
246.     //on demande la saisie du nom de la table
247.     if (strcmp(table_nom, t->nom) == 0) {
248.         //et si elle existe:
```



```
249.         scanf("%u", &rang_suppr);
250.         //on demande la saisie du rang de l'enregistrement à supprimer
251.         for (; rang_suppr <= t->nbEnrg; ++rang_suppr) {
252.             //on vérifie si l'enregistrement demandé existe et on décale les enregis-
trements à partir de rang_suppr jusqu'à table.nbEnrg
253.             for (unsigned int i = 0; i < t->nbChamps; ++i) {
254.                 //et pour i allant de 0 à nombre de champs-1
255.                 if (rang_suppr == t->nbEnrg) {
256.                     //lorsque on est à la dernière valeur de rang_suppr == table.nbE
nrg
257.                     --t->nbEnrg;
258.                     //on diminue le nombre de rangs de 1
259.                     return;
260.                 }
261.                 else {
262.                     strcpy(t->enrg[rang_suppr].colonne[i].data, t-
>enrg[rang_suppr + 1].colonne[i].data);
263.                     //sinon on supprime l'enregistrement demandé et on décale chaque
enregistrement d'un cran en arrière
264.                 }
265.             }
266.         }
267.         printf("enregistrement inconnu\n");
268.         //si l'enregistrement n'existe pas on affiche "enregistrement inconnu"
269.         return;
270.     }
271.     else
272.         printf("table inconnue\n");
273.     //sinon on affiche "table inconnue"
274. }
275. void afficher_enregistrement(const Table* t) {
276.     unsigned char table_nom[lgMot + 1];
277.     //variable locale de stockage du nom de la table dont on veut afficher un enregi-
strement
278.     unsigned int rang_enrg;
279.     //variable locale qui indique le rang de l'enregistrement à afficher
280.     scanf("%s", &table_nom);
281.     //on demande la saisie du nom de la table
282.     if (strcmp(table_nom, t->nom) == 0) {
283.         //et si elle existe
284.         scanf("%u", &rang_enrg);
285.         //on demande la saisie du rang de l'enregistrement à afficher
286.         if (rang_enrg <= t->nbEnrg) {
287.             //si l'enregistrement à afficher existe,
288.             printf("%u", rang_enrg);
289.             //on affiche son rang
290.             for (unsigned int i = 0; i < t->nbChamps; ++i) {
291.                 // pour i allant de 0 à nombre de champs-1
292.                 printf(" %s %s", t->schema[i].type, t-
>enrg[rang_enrg].colonne[i].data);
293.                 //on affiche les valeurs de chaque champ de l'enregistrement précédé
de sont type
294.             }
295.             printf("\n"); return;
296.             //on revient à la ligne et on sort de la boucle
297.         }
298.         printf("enregistrement inconnu\n");
299.         //si l'enregistrement n'existe pas on affiche "enregistrement inconnu"
300.         return;
301.     }
302.     else
303.         printf("table inconnue\n");
304.     //sinon on affiche "table inconnue"
305. }
306. void delete_table(Table* t) {
```




```
307.     unsigned char table_nom[lgMot + 1];
308.     //variable locale de stockage du nom de la table que l'on veut supprimer
309.     scanf("%s", &table_nom);
310.     //on demande la saisie du nom de la table
311.     if (strcmp(table_nom, t->nom) == 0) {
312.         //et si la table existe
313.         cmp_table = t->nbEnrg = t->nbChamps = 0;
314.         //on remet à zéro le nombre de champs, d'enregistrements et de table
315.         strcpy(t->nom, "\0");
316.     }
317.     else {
318.         printf("table inconnue\n");
319.         //sinon on affiche "table inconnue"
320.     }
321. }
322.
323. void select_enregistrement(const Table* t) {
324.     unsigned char table_nom[lgMot + 1], champ_nom[lgMot + 1];
325.     //variable locale de stockage du nom de la table
326.     scanf("%s", &table_nom);
327.     //on demande la saisie du nom de la table
328.     if (strcmp(table_nom, t->nom) == 0) {
329.         //et si la table existe
330.         scanf("%s", &champ_nom);
331.         //on demande la saisie du nom du champ
332.         scanf("%s %s", &crit1, &crit2);
333.         //on demande la saisie des critères de sélection
334.         for (unsigned int i = 0; i < t->nbChamps; ++i) {
335.             // pour i allant de 0 à nombre de champs-1
336.             if (strcmp(champ_nom, t->schema[i].nom) == 0) {
337.                 //quand on trouve que le champ existe
338.                 for (unsigned int rang_temp = 1; rang_temp <= t-
>nbEnrg; ++rang_temp) {
339.                     // pour rang_temp allant de 1 à nombre de table.nbEnrg
340.                     if (compare_enregistrement(t->schema[i].type, t-
>enrg[rang_temp].colonne[i].data)) {
341.                         //selon le résultat de compare_enregistrement on affiche ou
non la ligne d'enregistrement
342.                         printf("%u", rang_temp);
343.                         //on affiche son rang
344.                         for (unsigned int n = 0; n < t->nbChamps; ++n) {
345.                             // pour i allant de 0 à nombre de champs-1
346.                             printf(" %s %s", t->schema[n].type, t-
>enrg[rang_temp].colonne[n].data);
347.                             //on affiche l'enregistrement demandé précédé de sont ty
pe
348.                         }
349.                         printf("\n");
350.                         //on va à la ligne après avoir affiché un enregistrement
351.                     }
352.                 }
353.             }
354.         }
355.     }
356.     else {
357.         printf("table inconnue\n");
358.     }
359. }
360.
361. bool compare_enregistrement(const char type[lgMot + 1], const char data[lgMot + 1])
{
362.     if (strcmp(type, "INT") == 0) {
363.         //si data est de type INT
364.         return (atoi(crit1) <= atoi(data) && atoi(data) <= atoi(crit2) || atoi(crit2
) <= atoi(data) && atoi(data) <= atoi(crit1));
```



```
365.         //on renvoie TRUE(1) ou FALSE(0) si la valeur correspond aux critères
366.     }
367.     else if (strcmp(type, "FLOAT") == 0) {
368.         //si data est de type FLOAT
369.         return ((atof(crit1) <= atof(data)) && (atof(data) <= atof(crit2)) || (atof(
crit2) <= atof(data)) && (atof(data) <= atof(crit1)));
370.         //on renvoie TRUE(1) ou FALSE(0) si la valeur correspond aux critères
371.     }
372.     else if (strcmp(type, "TEXT") == 0) {         //si data est de type TEXT
373.         return ((crit1[0] <= data[0]) && (crit2[0] >= data[0]) || (crit2[0] >= data[
0]) && (crit1[0] <= data[0]));
374.         //on renvoie TRUE(1) ou FALSE(0) si la valeur correspond aux critères
375.     }
376.     else if (strcmp(type, "DATE") == 0) {
377.         //si data est de type DATE
378.         char aux[3][lgMot + 1];
379.         //variable temporaire lors de la transformation des dates
380.         unsigned int c1 = atoi(strncpy(aux[0], crit1, 2)) + atoi(strncpy(aux[0], cri
t1 + 3, 2)) * 100 + atoi(strncpy(aux[0], crit1 + 6, 4)) * 10000;
381.         // on transforme les critères et data
382.         unsigned int c2 = atoi(strncpy(aux[1], crit2, 2)) + atoi(strncpy(aux[1], cri
t2 + 3, 2)) * 100 + atoi(strncpy(aux[1], crit2 + 6, 4)) * 10000;
383.         // ex : les char 25/05/1999 -> un uint 19990525
384.         unsigned int data1 = atoi(strncpy(aux[2], data, 2)) + atoi(strncpy(aux[2], d
ata + 3, 2)) * 100 + atoi(strncpy(aux[2], data + 6, 4)) * 10000;
385.         return ((c1 <= data1 && data1 <= c2) || (c2 <= data1 && data1 <= c1));
386.         //on renvoie TRUE(1) ou FAUX(0) si la valeur correspond aux critères
387.     }
388.     else
389.         return 0;
390.     // si le type n'est pas reconnu on renvoie FALSE(0)
391. }
```



Trace d'exécution du sprint 5 :

```
Create_table Pays 5 nom TEXT pib INT capitale TEXT latitude FLOAT longitude FLOAT
Afficher_schema Pays
Pays 5
nom TEXT
pib INT
capitale TEXT
latitude FLOAT
longitude FLOAT
Insert_enregistrement Pays Turquie 10946 Ankara 39.92 32.86
Insert_enregistrement Pays Chine 6807 Pekin 39.92 116.44
Insert_enregistrement Pays Allemagne 45085 Berlin 52.52 13.41
Insert_enregistrement Pays Mexique 11180 Mexico 19.42 -99.12
Insert_enregistrement Pays Canada 51958 Ottawa 45.41 -75.69
Insert_enregistrement Pays Nigeria 3006 Lagos 6.45 3.39
Insert_enregistrement Pays Australie 67468 Melbourne -37.81 144.96
Insert_enregistrement Pays Bresil 11208 Brasilia -22.45 -49.23
Insert_enregistrement Pays Japon 38492 Tokyo 35.70 139.73
Insert_enregistrement Pays Zimbabwe 906 Harare -17.82 31.03
Insert_enregistrement Pays Russie 14555 Moscou 55.75 37.61
Insert_enregistrement Pays Maroc 3083 Rabat 33.97 -6.84
Insert_enregistrement Pays France 41421 Paris 48.85 2.35
Insert_enregistrement Pays Singapour 55182 Singapour 1.35 103.81
Afficher_enregistrements Pays
1 TEXT Turquie INT 10946 TEXT Ankara FLOAT 39.92 FLOAT 32.86
2 TEXT Chine INT 6807 TEXT Pekin FLOAT 39.92 FLOAT 116.44
3 TEXT Allemagne INT 45085 TEXT Berlin FLOAT 52.52 FLOAT 13.41
4 TEXT Mexique INT 11180 TEXT Mexico FLOAT 19.42 FLOAT -99.12
5 TEXT Canada INT 51958 TEXT Ottawa FLOAT 45.41 FLOAT -75.69
6 TEXT Nigeria INT 3006 TEXT Lagos FLOAT 6.45 FLOAT 3.39
7 TEXT Australie INT 67468 TEXT Melbourne FLOAT -37.81 FLOAT 144.96
8 TEXT Bresil INT 11208 TEXT Brasilia FLOAT -22.45 FLOAT -49.23
9 TEXT Japon INT 38492 TEXT Tokyo FLOAT 35.70 FLOAT 139.73
10 TEXT Zimbabwe INT 906 TEXT Harare FLOAT -17.82 FLOAT 31.03
11 TEXT Russie INT 14555 TEXT Moscou FLOAT 55.75 FLOAT 37.61
12 TEXT Maroc INT 3083 TEXT Rabat FLOAT 33.97 FLOAT -6.84
13 TEXT France INT 41421 TEXT Paris FLOAT 48.85 FLOAT 2.35
14 TEXT Singapour INT 55182 TEXT Singapour FLOAT 1.35 FLOAT 103.81
Delete_enregistrement Pays 10
Insert_enregistrement Pays Indonesie 3475 Jakarta -6.17 106.82
Delete_enregistrement Pays 1
Afficher_enregistrement Pays 10
10 TEXT Maroc INT 3083 TEXT Rabat FLOAT 33.97 FLOAT -6.84
Select_enregistrement Pays longitude -45 45
2 TEXT Allemagne INT 45085 TEXT Berlin FLOAT 52.52 FLOAT 13.41
5 TEXT Nigeria INT 3006 TEXT Lagos FLOAT 6.45 FLOAT 3.39
9 TEXT Russie INT 14555 TEXT Moscou FLOAT 55.75 FLOAT 37.61
10 TEXT Maroc INT 3083 TEXT Rabat FLOAT 33.97 FLOAT -6.84
11 TEXT France INT 41421 TEXT Paris FLOAT 48.85 FLOAT 2.35
Delete_table Pays
Create_table Calendrier 3 evenement DATE description TEXT pays TEXT
Afficher_schema Calendrier
Calendrier 3
evenement DATE
description TEXT
pays TEXT
Insert_enregistrement Calendrier 14/10/1066 Bataille_Hastings Angleterre
Insert_enregistrement Calendrier 19/08/1274 Sacre_Edouard_I Angleterre
Insert_enregistrement Calendrier 07/10/1337 Guerre_Cent_Ans France
Insert_enregistrement Calendrier 07/06/1494 Traite_Tordesillas Espagne
Insert_enregistrement Calendrier 13/09/1515 Bataille_Marignan France
Insert_enregistrement Calendrier 10/08/1539 Ordonnance_villiers_Cotterets France
Insert_enregistrement Calendrier 13/04/1598 Edit_Nantes France
Insert_enregistrement Calendrier 02/09/1666 Incendie_Londres Angleterre
Insert_enregistrement Calendrier 21/01/1793 Mort_Louix_XVI France
Insert_enregistrement Calendrier 28/05/1936 Machine_Turing Angleterre
Select_enregistrement Calendrier pays D G
3 DATE 07/10/1337 TEXT Guerre_Cent_Ans TEXT France
4 DATE 07/06/1494 TEXT Traite_Tordesillas TEXT Espagne
5 DATE 13/09/1515 TEXT Bataille_Marignan TEXT France
6 DATE 10/08/1539 TEXT Ordonnance_villiers_Cotterets TEXT France
7 DATE 13/04/1598 TEXT Edit_Nantes TEXT France
9 DATE 21/01/1793 TEXT Mort_Louix_XVI TEXT France
Select_enregistrement Calendrier evenement 07/10/1337 10/08/1539
3 DATE 07/10/1337 TEXT Guerre_Cent_Ans TEXT France
4 DATE 07/06/1494 TEXT Traite_Tordesillas TEXT Espagne
5 DATE 13/09/1515 TEXT Bataille_Marignan TEXT France
6 DATE 10/08/1539 TEXT Ordonnance_villiers_Cotterets TEXT France
```